

TP 14

Représentation des graphes – Sujet

Génération d'une grille

Soit une grille rectangulaire $n \times p$ constituée de n colonnes et de p lignes contenant toutes les arêtes possibles. On modélise cette grille par un graphe dont l'ensemble des sommets est donné par les couples (i, j) tels que : $i \in \llbracket 0, n \rrbracket$ et $j \in \llbracket 0, p \rrbracket$.

Les voisins d'un sommet (i, j) sont ceux situés en haut, en bas, à droite et à gauche s'ils existent (par exemple, le sommet $(0, 0)$ a comme voisin les sommets $(0, 1)$ et $(1, 0)$).

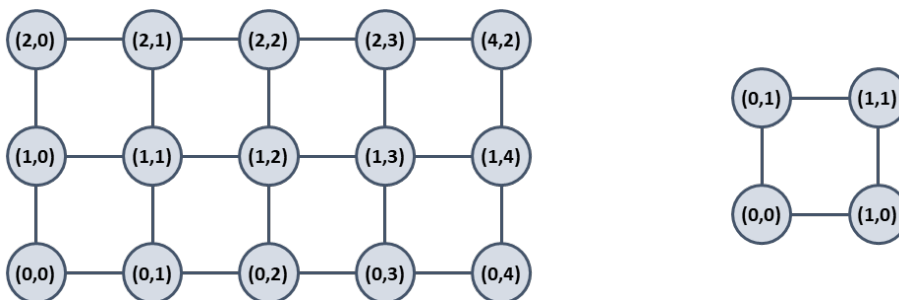


FIGURE 1 – Grille (5,3) et grille (2,2)

Le graphe est implémenté par un dictionnaire d'adjacence où les clés sont les tuples, coordonnées d'un sommet. La valeur associée est une liste des sommets voisins.

Question 1 Écrire la fonction `creer_graphe(n:int, p:int) -> dict` permettant de créer le graphe d'une grille de n colonnes et p lignes.

Exemple –

La grille 2×2 sera modélisée par le graphe suivant :

```
1 >>> G2 = creer_graphe(2,2)
2 >>> G2
3      {(0, 0): [(1, 0), (0, 1)],
4       (1, 0): [(1, 1), (0, 0)],
5       (0, 1): [(1, 1), (0, 0)],
6       (1, 1): [(0, 1), (1, 0)]}
```

On souhaite afficher ce graphe en utilisant `matplotlib`. Pour cela, on va commencer par tracer chacune des arêtes puis chacun des sommets.

Question 2 Écrire la fonction `get_sommets(G:{}) -> ([],[])` renvoyant deux listes `les_x` et `les_y` contenant respectivement les abscisses des sommets et les ordonnées des sommets.

Exemple –

Dans l'exemple qui suit, les coordonnées de sommets peuvent être dans un ordre différent.

```
1 >>> les_sx, les_sy = get_sommets(G2)
2 >>> les_sx, les_sy
3      ([0, 1, 0, 1], [0, 0, 1, 1])
```

Question 3 Écrire la fonction `trace_sommets(G:{}, couleur : str) -> None` qui trace sur la figure courante les sommets en utilisant un point coloré.

Exemples : pour tracer avec des points rouge on utilise la fonction suivante : `plt.plot(x,y,'r.')` ; en bleu : `plt.plot(x,y,'b.')`, en noir : `plt.plot(x,y,'k.')` .

Question 4 Écrire la fonction `get_aretes(G:{}) -> []` renvoyant la liste des arêtes du graphe sous la forme d'une liste de listes de tuples. Une arête est donc une liste de sommets où les sommets sont des tuples. Les arêtes ne devront être présentes qu'une fois.

Exemple –

Dans l'exemple qui suit, l'ordre des arêtes peut être dans un ordre différent. Pour une arête donnée, les sommets peuvent aussi être dans un ordre différent.

```
1 >>> get_aretes(G2)
2      [[(0, 0), (1, 0)], [(0, 0), (0, 1)], [(1, 0), (1, 1)], [(0, 1), (1, 1)]]
3      []
```

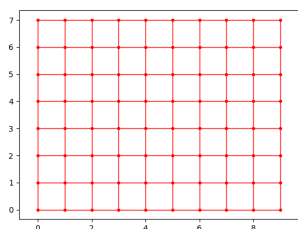


FIGURE 2 – Tracé d'un graphe grille de 10 colonnes et 8 lignes

Question 5 Écrire la fonction `trace_arete(s1:(), s2:(), couleur : str, epaisseur : int) -> None` qui trace une arête reliant les sommets `s1` et `s2` sur la figure courante.

Exemple : pour tracer l'arête `[(0,2),(1,2)]` en bleu avec une épaisseur de 2, il faut utiliser l'instruction : `plt.plot([0,1],[2,2],'b',linewidth=2)`.

Question 6 Écrire la fonction `trace_graphe(G:{},couleur : str,epaisseur : int) -> None` qui permet de tracer les sommets et les arêtes du graphe `G` sur la figure courante. Tracer le graphe en rouge avec une épaisseur de 1 pour obtenir la figure ci-dessous.

On définit un chemin comme une liste de sommets.

Question 7 Ecrire la fonction `verif_chemin(G:{}, chemin : [()]) -> bool` qui renvoie `True` si le chemin appartient au graphe, `False` sinon.

Question 8 Écrire la fonction `trace_chemin(G:{}, chemin:{}) -> None` qui permet de superposer le graphe et le chemin.

Graphe d'intervalles : Énigme de Claude BERGE

A l'époque, le Duc de Densmore avait été tué par l'explosion d'une bombe artisanale, qui avait également détruit le château de Densmore où il s'était retiré. Les journaux d'alors relataient que le testament, détruit lui aussi par l'explosion, avait tout pour déplaire à l'une de ses huit ex-femmes. Or, avant sa mort, le Duc les avait toutes invitées à passer quelques jours dans sa retraite écossaise.

Les suspects sont donc ces huit femmes présentes sur l'île au moment des faits. Chacune n'y ayant effectué qu'un seul séjour. L'une d'entre elles a dû se cacher à un moment propice pour aller préparer et poser la bombe.

Les faits étant assez anciens, les suspects se souviennent des personnes qu'elles ont croisées, mais pas dans quel ordre. Voici les témoignages :

- Ann a déclaré y avoir rencontré Betty, Cynthia, Emily, Felicia et Georgia ;
- Betty a déclaré y avoir rencontré Ann, Cynthia et Helen ;
- Cynthia a déclaré y avoir rencontré Ann, Betty, Diana, Émily et Helen ;

- ▶ Diana a déclaré y avoir rencontré Cynthia et Emily;
- ▶ Emily a déclaré y avoir rencontré Ann, Cynthia, Diana et Felicia;
- ▶ Felicia a déclaré y avoir rencontré Ann et Emily;
- ▶ Georgia a déclaré y avoir rencontré Ann et Helen;
- ▶ Helen a déclaré y avoir rencontré Betty, Cynthia et Georgia.

Précision supplémentaire, personne d'autre n'est venu sur l'île. Par ailleurs si deux femmes ne se sont pas vues, c'est soit qu'elles n'étaient pas l'île en même temps, soit que l'une d'entre elles était cachée dans le labyrinthe et préparait la bombe artisanale...

Question 9 L'inspecteur qui vous a précédé sur l'affaire a constaté que les témoignages concordent et avaient conclu à la culpabilité du majordome mort dans l'explosion avec le duc. Mais ce n'est pas le cas. Pouvez-vous trouver le coupable ?

Références : T. Kovaltchouk.

Déplacement d'un cavalier sur un échiquier

Un cavalier se déplace, lorsque c'est possible, de 2 cases dans une direction verticale ou horizontale, et de 1 case dans l'autre direction (le trajet dessine une figure en L).

Dans un premier temps, les cases de l'échiquier sont représentées par des tuples : le couple (i, j) désigne la case d'abscisse i et d'ordonnée j . Un échiquier possède 8 colonnes et 8 lignes, donc i et j seront compris entre 0 et 7.

Question 10 Écrire une fonction `estDansEch(i:int, j:int) -> bool` qui renvoie `True` si (i, j) correspond à une case valide de l'échiquier et `False` sinon.

Question 11 Écrire une fonction `mvtsPossibles(i:int, j:int) -> [(int,int)]` qui renvoie la liste des cases où le cavalier peut se déplacer à partir de la case (i, j) à l'ordre près.

Question 12 Vérifier que :

- ▶ `mvtsPossibles(0,0)` renvoie $[(1, 2), (2, 1)]$,
- ▶ `mvtsPossibles(3,5)` renvoie bien $[(1, 4), (1, 6), (2, 3), (2, 7), (4, 3), (4, 7), (5, 4), (5, 6)]$,
- ▶ `mvtsPossibles(7,7)` renvoie bien $[(5, 6), (6, 5)]$.

Tous ces résultats sont à l'ordre près.

Question 13 Créer un graphe `G` sous la forme d'un dictionnaire d'adjacence avec pour sommets les différentes cases de l'échiquier et les arêtes qui correspondent à un mouvement possible du cavalier.

Question 14 Vérifiez que vous avez un graphe avec 64 sommets et 168 arêtes.

Le codage des cases d'échiquier se fait classiquement par une chaîne de caractères comprenant : une lettre minuscule pour l'abscisse (de `a` à `h`) et un chiffre pour l'ordonnée (de 1 à 8). La case en bas à gauche de l'échiquier est donc de code `'a1'`.

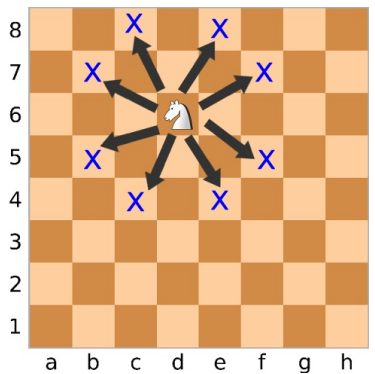
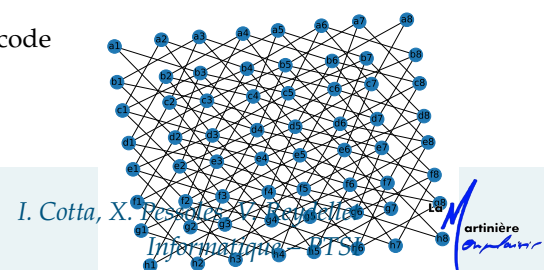


FIGURE 3 – Illustration du mouvement d'un cavalier sur un échiquier

Remarque

`ord(c)` retourne le codage Unicode correspondant au caractère `c` et `chr(n)` renvoie le caractère dont le codage Unicode est `n`.

Question 15 Écrire une fonction `codage(i:int, j:int) -> str` qui renvoie le code correspondant à la case (i, j) .



Question 16 Créer un dictionnaire d'adjacence `G2` comme défini précédemment avec les cases maintenant nommées d'après leur code.

Question 17 Vérifier que, à l'ordre près :

- ▶ `G2['a1']` renvoie `['b3', 'c2']`,
- ▶ `G2['d6']` renvoie bien `['b5', 'b7', 'c4', 'c8', 'e4', 'e8', 'f5', 'f7']`,
- ▶ `G2['h8']` renvoie bien `['f7', 'g6']`.